

Clock Tree Synthesis with CCOpt

For a visual demonstration of the material in this document, please see the tutorials at:

- Simple block CTS: https://youtu.be/8_syH8kP_1I
- Full SoC CTS: <https://youtu.be/bD04HLDIDy0>

1 Introduction

Clock Tree Synthesis is done with a tool known as Clock Concurrent Optimization, or CCOpt. The CCOpt tool runs a simultaneous clock tree synthesis and timing optimization engine with a very different approach to traditional zero-skew based CTS.

Note that prior to version 14 or so of Encounter, the clock tree synthesis tool was simply known as “CTS” (or FE-CTS). This tool used a different engine and the configurations were in their own proprietary syntax. The CTS tool is still provided for backwards compatibility, and is used for buffer tree insertion; however, Cadence recommends using CCOpt. In addition, CCOpt uses TCL based configuration commands and comes with the Clock Tree Debugger, which is a huge improvement on previous debugging tools.

1.1 Clock Tree Synthesis Engines

Since not everyone is ready for this change of methodology and not every design fits the approach of CCOpt, Cadence provides two additional modes for performing CTS: CCOpt-CTS and legacy FE-CTS. To invoke each one of these options, use the following commands:

- CCOpt: concurrent clock optimization (with the CCOpt engine)

```
cchopt_design
```

- CCOpt-CTS: global skew balanced CTS with the CCOpt engine

```
clock_design
```

Note that running **cchopt_design** concurrently optimizes the timing of the design. On the other hand, running **clock_design** requires running **opt_design -post_cts** afterward in order to optimize the timing.

The concurrent clock optimization engine heavily applies useful skew. However it will not deskew a sink by more than a preset factor called **auto_limit_insertion_delay** factor that is defaulted to 1.5.

1.2 Clock Tree Spec

Running clock tree synthesis, in a similar fashion to running logic synthesis, requires the definition of constraints for the tool. In legacy FE-CTS, these constraints were defined in a special file with its own

proprietary syntax. Luckily, with the introduction of CCOpt, Cadence made the clock tree synthesis commands and constraints standard TCL functions that can be entered in the Innovus shell. However, it is still convenient to define the CCOpt commands in a separate spec file.

1.2.1 Creating an initial CCOpt Spec File

A default version of such a spec can be generated automatically, according to Innovus's knowledge of your design.

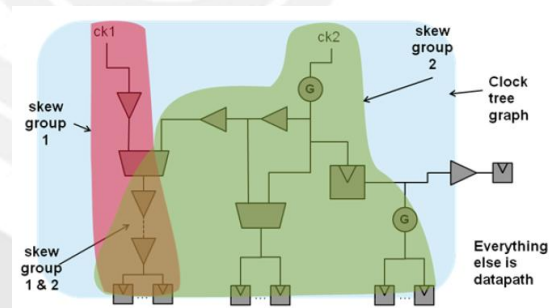
To create an automatic CCOpt spec file (mainly based on your SDC), use the following command:

```
create__clock_tree_spec -out_file <filename>
```

1.3 Clock Tree and Skew Groups

CCOpt differentiates between *clock trees* and *skew groups*. A clock tree is basically all the clock sinks (register clock pins, macro clock pins) driven by a clock source (output pin of an instance or input port to the design). On the other hand, a skew group is a balancing constraint on a set of sinks.

To make this clearer, the default is that all sinks driven by a clock tree are in a single skew group, and therefore, they are skew constrained to each other. However, if necessary, a clock tree can have several skew groups and the sinks belonging to each group will be skew constrained separately. When running [clock_design](#), it will balance all the sinks that belong to a certain skew group (as opposed to a certain clock tree).



Note that skew groups can overlap, i.e., share sources and/or sinks.

1.3.1 Properties of Clock Trees and Skew Groups

In general, most target constraints and properties are set globally for all clock trees and skew groups. However, certain constraints and properties can be set separately for a specific clock tree or skew group, as follows (these properties will be explained later):

- Clock Trees: Maximum Transition Time, Route Types
- Skew Groups: Skew Target, Insertion Delay, ignore pins

1.3.2 Defining Clock Trees

To define a skew group, use the [create_clock_tree](#) command:

```
create_clock_tree -name <clock tree name> -source <source pin> \
[-no_skew_group]
```

The [-no_skew_group](#) option disables the automatic creation of a corresponding skew group.

To define a generated clock:

```
create_generated_clock_tree -name <name> -source <source pin> \
                             -generated_by <source pin>
```

1.3.3 Defining Skew Groups

The usual way to define a skew group is with the `-auto_sinks` option, which will add all the sinks of the clock source to the skew group.

```
create_skew_group -name <skew grp name> -sources <clock name> \
                  -auto_sinks
```

Of course, other options of the command enable specific definitions for which pins are in the skew group and what the target skew, insertion delay, etc. are. You can also add or remove sinks and ignore pins with the `update_skew_group` command.

1.4 Clock Tree Sink Pin Types

1.4.1 Ignore Pins

An ignore pin is considered as a part of the clock tree graph. CTS will perform DRV buffering up to the pin, but the pin will not be considered as a sink in any skew group, which means the latency to an ignore pin is not important. Tracing through and beyond the pin will be disabled.

```
set_db pin:<pin_name> .cts_sink_type ignore
```

An example reason for using an ignore pin, is to disable a clock path in a certain operating mode. Remember, we are using MMMC, so we can define the clock tree for multiple modes with different SDCs, and an ignore pin can drive the clock tree synthesis according to case analyses set in different modes. Basically, if each mode uses a separate input of a multiplexer, then two skew groups would be defined – one for each mode, and each skew group would have different ignore pins. For example:

```
create_clock_tree -name clk -source clk -no_skew_group
create_skew_group -name clk/mode0 -source clk -auto_sinks
create_skew_group -name clk/mode1 -source clk -auto_sinks
update_skew_group clk/mode0 -add_ignore_pins mux/I0
update_skew_group clk/mode1 -add_ignore_pins mux/I1
```

1.4.2 Stop Pins

A stop pin is considered as a part of the clock tree graph. CTS will perform DRV buffering up to the pin and by default the pin will be considered a sink to be balanced in any skew group that reaches the stop pin. Tracing through and beyond the pin will be disabled.

```
set_db pin:<pin_name> .cts_sink_type stop
```

1.4.3 Exclude Pins

An exclude pin is a pin that is not part of the clock tree graph but might still be connected to a clock net anyway if the same net has other clock fanout. Specifically, the clock tree graph must not extend beyond an exclude pin but it can be pruned back from an exclude pin.

```
set_db pin:<pin_name> .cts_sink_type exclude
```

1.5 Clock Net Types and Routing Rules for CTS

CCOpt differentiates between three categories of nets in a hierarchical fashion:

1. **Top Nets:** The first nets of a clock tree that drive many sinks (you can define which ones).
2. **Trunk Nets:** Any clock tree net that is not connected to clock sinks.
3. **Leaf Nets:** Any net that is connected to one or more clock sinks.

The use of the clock net categories is to define routing types according to clock net category using the `create_route_type` command. For example, you can decide that top nets are routed in high thick metals and are shielded and leaf nets are double spaced but not shielded.

1.5.1 Net Type and Routing Rule Settings

Sometimes, you will want to make special routing rules for clock nets, such as preferred routing metals, double spacing, and/or shielding. This is achieved through a combination of the `create_route_type` and `set_ccopt_property` commands. Note that special design rules, such as double spacing, are defined with *non-default rules* (NDRs). These can either be defined in the LEF or with the `create_route_rule` command.

The following example demonstrates a use case. Top clock route is defined with double width, double spacing (CTS_2W2S non-default rule), is routed in M8/M9, and is shielded by ground in M8. Trunks are also 2W2S but are routed in M6/M7 with ground shielding in M6, and routes to clock leafs are double width, single space (CTS_2W1S) in M4/M5 with no shield:

```
create_route_type -name leaf_rule -non_default_rule CTS_2W1S \
    -top_preferred_layer M5 -bottom_preferred_layer M4
set_db cts_route_type_leaf leaf_rule

create_route_type -name trunk_rule -non_default_rule CTS_2W2S \
    -top_preferred_layer M7 -bottom_preferred_layer M6 \
    -shield_net VSS -bottom_shield_layer M6
set_db cts_route_type_trunk trunk_rule

create_route_type -name top_rule -non_default_rule CTS_2W2S \
    -top_preferred_layer M9 -bottom_preferred_layer M8 \
    -shield_net VSS -bottom_shield_layer M8
set_db cts_route_type_top top_rule
```

You actually define which nets are trunk nets and which are top nets with the `cts_top_fanout_threshold` property. So to define that all nets that have at least 10000 downstream sinks will be considered top nets and not trunk nets, you would use:

```
set_db cts_top_fanout_threshold 10000
```

1.5.2 Routing Rule Recommendations

The recommendations for routing rule configurations are:

1. Top Nets and Trunk Nets:
 - a. Double width – to reduce resistance and use big vias
 - b. Double spacing – to reduce the capacitance caused by shielding
 - c. Shielding – to avoid aggressors that would impact timing
 - d. Middle to High Metal Layers - use a single pair of layers, one horizontal, one vertical, which have same pitch, width, and spacing rules.
2. Leaf Nets:
 - a. Double width – to reduce resistance
 - b. Middle routing layers - use a single pair of layers, one horizontal, one vertical, which have same pitch, width, and spacing rules.
 - c. Optional – Double spacing and/or shielding, but this may be infeasible

1.6 Library Cells for CTS

CCOpt allows you to define specific cells to use to build the clock tree, divided into four categories:

1. Non-Inverting Buffers
2. Inverters
3. Integrated Clock Gating Cells
4. Logic cells

1.6.1 Library Cell Definition Commands

- Define cells for building the clock tree:

```
set_db .cts_buffer_cells {bufA bufB bufC}
set_db .cts_inverter_cells {invA invB invC}
set_db .cts_clock_gating_cells {cgA cgB cgC}
set_db .cts_logic_cells {ckmux cknand cknor}
```

- Tell CCOpt to build the clock tree only using inverters (and not buffers):

```
set_db .cts_use_inverters true
```

1.6.2 Library Cell Recommendations for CTS

Recommendations for choosing clock tree cells are:

- Use Low VT (LVT) cells – reduces insertion delay (less OCV)

- Use inverters rather than buffers – reduces insertion delay
- Do not use the largest library cells – can cause EM problems and high power
- Do not use small cells (< X3) – poor cross-corner scaling
- Reduce run time by providing only ~5 cells per type

1.7 CTS Targets and Constraints

1.7.1 Transition Target

Define the maximum transition target (in ps, ns, etc.)

```
set_db .cts_target_max_trans <value>
```

This can be separately configured for different net types.

If a set_max_transition is defined in the SDC, it will be translated into a target_max_trans_sdc property:

```
SDC: set_max_transition 0.123 [get_clocks clk]
CTS: set_db clock_tree:clk .cts_target_max_trans_sdc 0.123
```

1.7.2 Skew Target

Define a skew target (**for clock_design only!**):

```
set_db .cts_target_skew 50ps ; # only for clock_design option
```

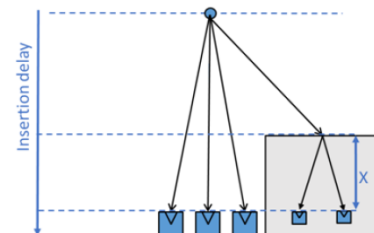
1.8 Defining CTS Specifications

1.8.1 Stop and Ignore Pins

Stop pins and ignore pins both stop the clock tree specification process from tracing beyond a pin. A stop pin is treated as a sink to be balanced whereas an ignore pin is not balanced.

1.8.2 Macro Clock Inputs

Since macros have internal clock latency, you will often want to arrive at the clock input of the macro early in order to balance the macro with the rest of the design. This is done by defining the clock insertion delay *inside the macro* with the `insertion_delay` property (see `x` in the figure):



```
set_db pin:mem1/CK .cts_insertion_delay 1.2ns
```

1.9 Post-CTS Timing Constraints

Timing constraints (SDC) are defined during synthesis considering an ideal clock. Therefore, traditional approaches load a new SDC file after CTS. The major changes include:

- Removal of `set_ideal_network` from the clock networks.
- Initiation of `set_propagated_clocks` to propagate the clocks through the network.
- Updating the timing constraints of the I/O ports to reflect insertion delays.

When using the CCOpt engine (both CCOpt and CCOpt-CTS) this is basically taken care of for you automatically. The clocks are automatically propagated and the source latencies are updated to account for the insertion delays to the I/Os.

WARNING: When using the CCOpt engine DO NOT use `set_propagated_clocks` and do not manually update I/O constraints after CTS.

Note that CCOpt is fully MMMC-aware. The first defined setup analysis view is used to build the clock (primary CTS delay corner), but setup and hold constraints are checked for all selected analysis views (defined by the `set_analysis_view` command).

1.10 Reporting and debugging the clock tree

1.10.1 Before running CCOpt

```
ccopt_check_prerequisites
```

1.10.2 CCOpt Tree Debugger

The best way to start is by opening the CCOpt Tree Debugger GUI.

Choose **CLOCK → CCOPT CLOCK TREE DEBUGGER**.

1.10.3 Skew Group Properties

- Find all the sink members of a skew group:

```
get_ccopt_property -skew_group <name> sinks
```

- Find all skew groups that a pin is a member of:

```
get_ccopt_property -pin <name> skew_groups_sink
```

-

1.10.4 Clock Tree and Skew Group Reports

- Report the entire clock tree:

```
report_ccopt_clock_trees -filename <file>
report_ccopt_clock_trees -summary -filename <file>
report_ccopt_clock_tree_structure -clock_tree <clock name>
get_ccopt_clock_tree_cells
get_ccopt_clock_tree_nets
get_ccopt_clock_tree_sinks
```

- Report the skew groups:

```
report_ccopt_skew_groups -filename <file>
```

```
get_ccopt_skew_groups
```

- Get the insertion delay for a skew group or a particular sink:

```
get_ccopt_skew_group_delay
```

- Report clock nodes along Worst Negative Slack path:

```
report_ccopt_worst_chain
```

1.10.5 Check Clock Tree Convergence

To check if the clock tree undesirably propagates into the datapath, run:

```
check_ccopt_clock_tree_convergence
```

```
report_ccopt_clock_tree_convergence
```

This document was provided to you courtesy of Prof. Adam Teman of the EnICS Labs Institute in the Alexander Kofkin Faculty of Engineering at Bar-Ilan University. The content was entirely written and compiled by Prof. Teman (without the help of AI tools), based on knowledge and non-proprietary data. You can find all of Prof. Teman's lectures and materials at the EnICS Labs website:

<https://enicslabs.com/education/>

For any comments or enquiries, Prof. Teman can be reached at adam.teman@biu.ac.il